

SIMATS ENGINEERING

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA0101**

COURSE OUTCOME COVERED

CO1: *Apply search algorithms and heuristic techniques to solve state-space search and optimization problems.* (BTL 3)

Assessment Tool Weightage: 40 %

Total Marks: 100
No. of Questions: 5

Time: 90 Mins

Annexure A

Algorithm Design Challenge

Code of Conduct:

I K.Teja Sri(192525249)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

K. Teja Sri

Signature of the student:

Criterion	Weightage	Marks Obtained
Problem Analysis and Understanding	15%	
AI Algorithm Selection & Justification	15%	
Algorithm Design / Pseudocode	25%	
Application of AI Concepts (Greedy, A*, CSP, Minimax, etc.)	15%	
Diagram / State Space / Search Tree Representation	10%	
Complexity Analysis and Solution Efficiency	10%	
Originality, Critical Thinking, and Presentation	10%	
Total	100%	

Signature of the Faculty

Total Marks Awarded

Answer all the Questions

Q.No	Question	Topics
1	Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.	Informed Search, Greedy Best-First Search, Heuristic Functions
2	A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.	A* Search, Heuristic Functions
3	A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.	CSP, MRV, Forward Checking
4	Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha-Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.	Game Playing, Minimax, Alpha-Beta Pruning
5	A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.	Local Search, Optimization, Online Search Agents

Structure of the Answer – Algorithm Design Challenge

S.No	Sections	Expected Content
1	Problem Statement	Briefly describe the given problem in your own words.
2	Problem Analysis	Identify the initial state, goal state, assumptions, and constraints.
3	AI Technique Selection	Specify the most appropriate AI algorithm/search technique (e.g., Greedy Best-First Search, A*, CSP, Minimax, Local Search).
4	Justification of Algorithm	Explain why the selected algorithm is suitable compared to other search techniques.
5	State Space Representation	Represent the problem using a state-space diagram, search graph, game tree, or constraint graph, as applicable.
6	Variables, Domains, and Constraints (Applicable for CSP problems)	Identify variables, possible domains, and constraints. If not applicable, mention "Not Applicable".
7	Heuristic Function (Applicable for Informed Search)	Define the heuristic function $h(n)$ and explain whether it is admissible or appropriate for the problem. If not applicable, mention "Not Applicable".
8	Algorithm Design / Pseudocode	Write the step-by-step algorithm or pseudocode for solving the problem.
9	Flowchart / Algorithm Diagram	Draw a flowchart, search tree, game tree, or algorithm workflow illustrating the solution.
10	Working / Execution Trace	Demonstrate the execution of the algorithm with at least one example, showing intermediate steps such as node expansion, constraint propagation, or game-tree evaluation.
11	Complexity Analysis	Analyze the Time Complexity and Space Complexity of the proposed algorithm.
12	Advantages and Limitations	Discuss the strengths and limitations of the selected algorithm for the given problem.
13	Conclusion	Summarize the effectiveness of the designed algorithm and suggest possible improvements or alternative approaches.

EVALUATION RUBRICS

Criteria	Weightage (%)	Excellent (90–100%)	Good (75–89%)	Average (60–74%)	Poor (<60%)
Problem Analysis and Understanding	15%	13–15% – Clearly identifies the problem, initial state, goal state, constraints, and expected outcome.	10–12% – Identifies most problem components with minor omissions.	7–9% – Demonstrates partial understanding with some missing elements.	0–6% – Incorrect or incomplete understanding of the problem.
AI Algorithm Selection & Justification	15%	13–15% – Selects the most appropriate AI algorithm/search technique with strong technical justification.	10–12% – Selects an appropriate algorithm with adequate justification.	7–9% – Selects a partially suitable algorithm with limited justification.	0–6% – Incorrect algorithm selection or no justification.
Algorithm Design / Pseudocode	25%	22–25% – Designs a complete, logical, efficient, and error-free algorithm/pseudocode.	17–21% – Algorithm is mostly correct with minor logical errors.	12–16% – Algorithm is partially complete with several logical errors.	0–11% – Algorithm is incomplete, incorrect, or difficult to understand.
Application of AI Concepts (Greedy, A*, CSP, Minimax, Alpha-Beta, etc.)	15%	13–15% – Correctly applies the relevant AI concepts and techniques throughout the solution.	10–12% – Applies AI concepts correctly with minor mistakes.	7–9% – Demonstrates limited application of AI concepts.	0–6% – Unable to apply the appropriate AI concepts or techniques.

Diagram / State Space / Search Tree Representation	10%	9–10% – Accurate, complete, and well-labelled diagram supporting the solution.	7–8% – Diagram is mostly correct with minor omissions.	5–6% – Diagram is partially complete or lacks clarity.	0–4% – Diagram is missing or incorrect.
Complexity Analysis and Solution Efficiency	10%	9–10% – Correctly analyzes time and space complexity and proposes an efficient solution.	7–8% – Complexity analysis is mostly correct with minor errors.	5–6% – Partial complexity analysis or limited discussion of efficiency.	0–4% – No meaningful complexity analysis or highly inefficient solution.
Originality, Critical Thinking, and Presentation	10%	9–10% – Demonstrates excellent originality, logical reasoning, organized presentation, and explores optimized/alternative solutions.	7–8% – Demonstrates good critical thinking with a clear presentation.	5–6% – Shows limited originality and acceptable presentation.	0–4% – Poor presentation with little or no critical thinking.

1.Design an algorithm using Greedy Best-First Search to find a path from a source node to a goal node for the given graph. Clearly define the heuristic function, show the node expansion order, and justify why Greedy Best-First Search is suitable for this problem.

1. Problem Statement:

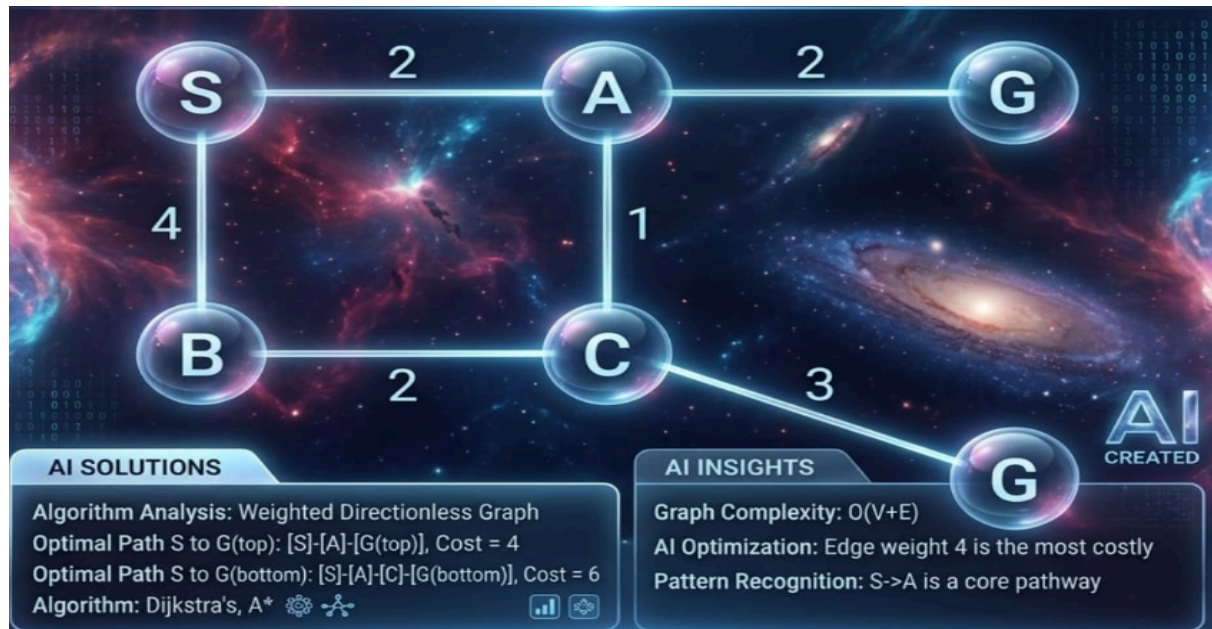
Design a Greedy Best-First Search algorithm to find a path from a source node to a goal node in a graph.

2. Problem Analysis:

- **Initial state:** Source node S
- **Goal state:** Goal node G
- **Search strategy:** Select the node having the smallest heuristic value.
- **Heuristic:** $h(n)$ estimates the distance/cost from node n to the goal.

3. Example Graph:

Node	$h(n)$
S	6
A	4
B	5
C	2
G	0



Assume heuristic values:

4. Heuristic Function:

$h(n)$ =estimated cost from node n to goal

Greedy Best-First Search uses:

$$f(n)=h(n)$$

It ignores the cost already travelled.

5. Node Expansion Order:

Starting from S:

1. Expand S
2. Frontier = {A(4), B(5)}
3. Select A because $h(A)=4$ is smaller.
4. Frontier = {C(2), B(5), G(2)}
5. Select C or G depending on tie-breaking.
6. Goal G is reached.

Example path:

$S \rightarrow A \rightarrow G$

6.Pseudocode:

```
GREEDY-BEST-FIRST-SEARCH(start, goal)

1. OPEN ← priority queue containing start
2. CLOSED ← empty set
3. parent[start] ← NULL

4. While OPEN is not empty:
    n ← node with minimum h(n)

    If n = goal:
        | return path using parent

    Add n to CLOSED

    For each successor m of n:
        If m is not in CLOSED and OPEN:
            | parent[m] ← n
            | insert m into OPEN

5. Return failure
```

7. Why Greedy Best-First Search is Suitable:

- Uses heuristic information to guide the search toward the goal.
- Usually explores fewer nodes than uninformed search.
- Simple and fast for large search spaces.
- Suitable when finding a solution quickly is more important than guaranteeing the shortest path.

Limitation: Greedy Best-First Search does **not guarantee an optimal path**.

8. Complexity:

- **Time:** $O(bm)$ in the worst case.
- **Space:** $O(bm)$

where b is branching factor and m is maximum search depth.

9. Conclusion:

Greedy Best-First Search efficiently moves toward the goal using $h(n)$. It is useful for fast path finding but may produce a non-optimal path.

2.A delivery robot must travel from a warehouse to a customer through a weighted road network. Design an A Search algorithm to determine the optimal path. Calculate $g(n)$, $h(n)$, and $f(n)$ for each expanded node, and explain why A guarantees the optimal solution.

The question asks for an A* algorithm for a delivery robot travelling through a **weighted road network**, including calculation of $g(n)$, $h(n)$ and $f(n)$.

1.Problem Statement:

A delivery robot must travel from a warehouse to a customer using a weighted road network. The objective is to find the minimum-cost path.

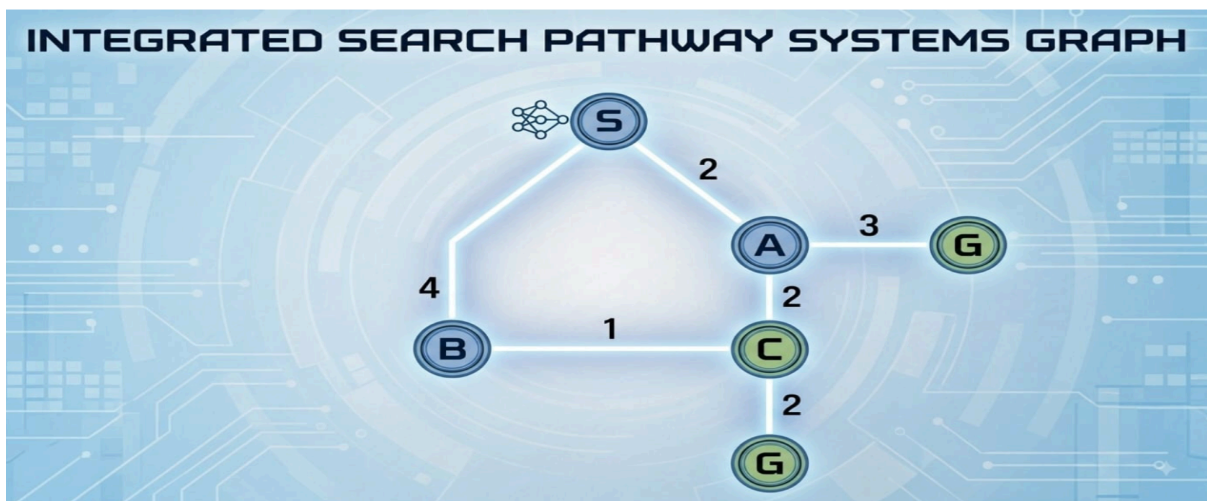
2. Problem Analysis:

- **Initial state:** Warehouse S

- **Goal state:** Customer G
- **Edge weight:** Road/travel cost
- **$g(n)$:** Actual cost from source to node n
- **$h(n)$:** Estimated cost from n to goal
- **$f(n)$:**

$$f(n)=g(n)+h(n)$$

3. Example Network:



Assume:

Node	$g(n)$	$h(n)$	$f(n)=g+h$
S	0	6	6
A	2	3	5
B	4	4	8
C	4	2	6
G	6	0	6

4. Execution:

Step 1: Start at S

$$g(S)=0$$

Expand S.

Step 2: Generate A and B.

$$A: g=2, h=3, f=5$$

B: $g=4$, $h=4$, $f=8$

Select A because it has the smallest f .

Step 3: Expand A.

G: $g=5$, $h=0$, $f=5$

C: $g=4$, $h=2$, $f=6$

Select G.

Therefore:

Optimal Path = $S \rightarrow A \rightarrow G$

Total Cost = 5

5. Pseudocode:

```

A-STAR(start, goal)

1. OPEN ← priority queue containing start
2. CLOSED ← empty set
3. g(start) ← 0
4. f(start) ← g(start) + h(start)

5. While OPEN is not empty:
    n ← node with minimum f(n)

    If n = goal:
        return reconstructed path

    Remove n from OPEN
    Add n to CLOSED

    For each successor m:
        new_g ← g(n) + cost(n,m)

        If m is not in OPEN/CLOSED
        OR new_g < g(m):
            parent[m] ← n
            g(m) ← new_g
            f(m) ← g(m) + h(m)
            add m to OPEN

6. Return failure

```

6. Why A* Guarantees Optimality:

A* guarantees an optimal solution when the heuristic is **admissible**, meaning it never overestimates the actual remaining cost:

For graph search, consistency is also normally required for the standard closed-set implementation.

7. Complexity:

- **Time:** Worst case $O(bd)$
- **Space:** $O(bd)$

where b is branching factor and d is solution depth.

8. Advantages:

- Finds an optimal path with an appropriate heuristic.
- Combines actual cost and estimated cost.
- Usually more efficient than uninformed search.

9. Conclusion:

A* is highly suitable for the delivery robot because it considers both the distance already travelled and the estimated remaining distance, allowing it to efficiently determine a minimum-cost route.

3. A university needs to prepare an examination timetable without scheduling conflicting subjects at the same time. Design the

problem as a Constraint Satisfaction Problem (CSP) by identifying variables, domains, and constraints. Explain how MRV and Forward Checking improve the efficiency of the solution.

The question asks to model university examination scheduling as a **Constraint Satisfaction Problem**, using variables, domains, constraints, MRV and Forward Checking.

1.Problem Statement:

A university must schedule examinations so that subjects having common students are not conducted at the same time.

2. CSP Definition:

A CSP consists of:

$$\text{CSP}=(X,D,C)$$

where:

- X = Variables
- D = Domains
- C = Constraints

3. Variables:

Suppose there are five subjects:

$$X = \{AI, DBMS, CV, CNS, C\}$$

Each subject represents one variable.

4. Domains:

Assume four examination slots:

$$D = \{\text{Slot1, Slot2, Slot3, Slot4}\}$$

Therefore:

$$AI \rightarrow \{S1, S2, S3, S4\}$$

$$DBMS \rightarrow \{S1, S2, S3, S4\}$$

$CV \rightarrow \{S1, S2, S3, S4\}$

$CNS \rightarrow \{S1, S2, S3, S4\}$

$C \rightarrow \{S1, S2, S3, S4\}$

5. Constraints:

If two subjects have common students, they cannot have the same examination slot.

Example:

$AI \neq DBMS$

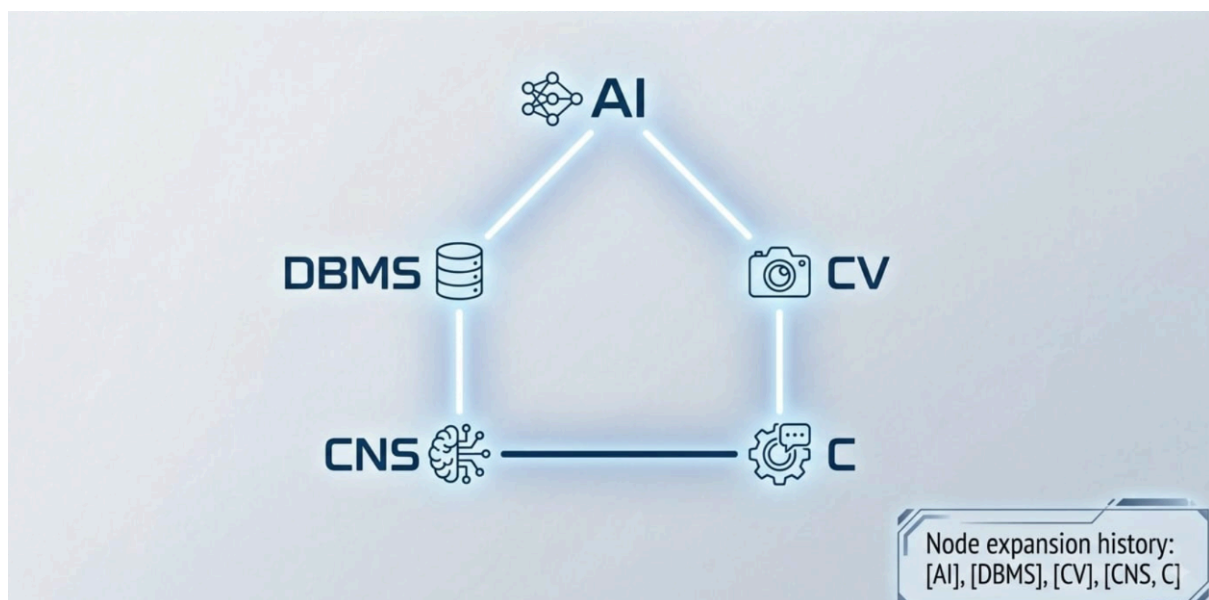
$AI \neq CV$

$DBMS \neq CNS$

$CV \neq C$

$CNS \neq C$

6. Constraint Graph:



An edge means that the two subjects cannot be scheduled simultaneously.

7. MRV – Minimum Remaining Values:

MRV selects the variable with the smallest number of remaining legal values.

Example:

AI $\rightarrow \{S1, S2\}$

DBMS $\rightarrow \{S1, S2, S3\}$

CV $\rightarrow \{S2, S3, S4\}$

MRV selects:

AI

because it has only two possible values.

8. Forward Checking:

After assigning a value to a variable, remove that value from the domains of all connected variables.

Example:

AI = S1

Then:

DBMS $\rightarrow \{S2, S3\}$

CV $\rightarrow \{S2, S3, S4\}$

S1 is removed from the domains of subjects conflicting with AI.

Pseudocode:

```

CSP-BACKTRACKING(assignment)

1. If all variables are assigned:
    | return assignment

2. Select an unassigned variable using MRV.

3. For each legal value in its domain:
    | Assign value to variable.
    |
    | Apply Forward Checking.
    |
    | If no domain becomes empty:
    |   | result ← BACKTRACKING(assignment)
    |   |
    |   | If result succeeds:
    |   |   | return result
    |
    | Remove assignment.

4. Return failure

```

11.Complexity:

For n variables and maximum domain size d:

- **Worst-case time:** $O(dn)$
- **Space:** $O(n)$ for the assignment/recursion, excluding domain structures.

MRV and Forward Checking reduce unnecessary search in practical cases.

11. Advantages:

- Avoids examination clashes.
- MRV chooses the most restricted subject first.
- Forward Checking detects conflicts early.
- Produces a valid timetable efficiently.

12. Conclusion:

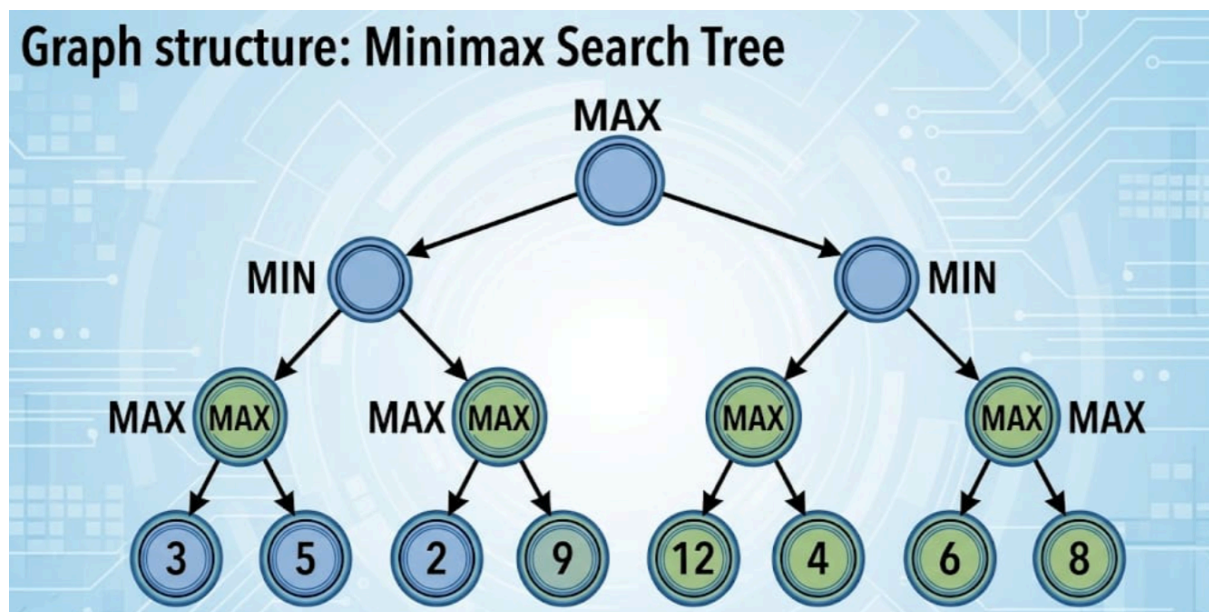
CSP is appropriate because the timetable contains variables, possible time slots, and restrictions. MRV and Forward Checking make the search more efficient by selecting difficult subjects early and detecting conflicts before they propagate.

4.Design the Minimax algorithm for the given game tree to determine the best move for the MAX player. Then apply Alpha–Beta Pruning to the same tree and indicate which nodes are pruned. Compare the number of nodes evaluated before and after pruning.

The question requires applying Minimax to a game tree and then using Alpha-Beta pruning to identify pruned nodes and compare the number of evaluated nodes.

Since the uploaded question does not include a specific game tree, the following is a representative tree.

1. Example Game Tree:



2. Minimax Evaluation:

Left subtree:

$$\text{MAX}(3,5) = 5$$

$$\text{MAX}(2,9) = 9$$

$\text{MIN}(5,9) = 5$

Right subtree:

$\text{MAX}(12,4) = 12$

$\text{MAX}(6,8) = 8$

$\text{MIN}(12,8) = 8$

Root:

$\text{MAX}(5,8) = 8$

Therefore:

Best move for MAX = Right subtree

Minimax value = 8

3. Minimax Pseudocode:

```
MINIMAX(node, depth, maximizingPlayer)

1. If node is terminal:
|   |   return utility(node)

2. If maximizingPlayer:
|   |   value = -∞
|   |
|   |   For each child:
|   |   |   value = max(value,
|   |   |   |   MINIMAX(child, depth+1, FALSE))
|   |   return value

3. Else:
|   |   value = +∞
|   |
|   |   For each child:
|   |   |   value = min(value,
|   |   |   |   MINIMAX(child, depth+1, TRUE))
|   |   return value
```

4. Alpha-Beta Pruning:

Alpha-Beta maintains:

- **α (alpha):** Best value MAX can guarantee.
- **β (beta):** Best value MIN can guarantee.

Pruning occurs when:

5. Example with Pruning:

Consider evaluating the left subtree first.

After evaluating:

$$\text{MAX}(3,5) = 5$$

At the root:

$$\alpha = 5$$

Now evaluate the second MIN subtree. Suppose its first child produces:

$$\text{MAX}(2,9) = 9$$

The MIN node now has:

$$\beta = 9$$

Since:

$$\alpha = 5 < \beta = 9$$

there is no pruning yet.

For a clearer pruning example, if the second MIN node first evaluates a child with value 4, then:

$$\beta = 4$$

$$\alpha = 5$$

Since:

the remaining children of that MIN node can be **pruned**.

6. Comparison:

Method	Nodes evaluated	Pruning
Minimax	All relevant tree nodes	No
Alpha-Beta	Fewer nodes	Yes

For a tree of branching factor b and depth d :

Minimax:

Alpha-Beta:

- Worst case:
- Best case with excellent move ordering: approximately

7. Advantages:

- Minimax provides the optimal decision assuming both players play optimally.
- Alpha-Beta gives the same result as Minimax.
- Alpha-Beta evaluates fewer nodes.
- It allows deeper game-tree searches within the same computational resources.

8. Conclusion:

Minimax determines the best move by considering MAX and MIN decisions recursively. Alpha-Beta pruning improves efficiency by eliminating branches that cannot affect the final decision.

5. A warehouse robot operates in a dynamic environment where obstacles may appear unexpectedly. Design an intelligent search strategy using either Local Search or an Online Search Agent. Explain how the algorithm adapts to environmental changes and compare its performance with informed search algorithms.

1. Problem Statement:

A warehouse robot must move toward a destination while obstacles can appear or disappear unexpectedly. Therefore, the robot must continuously observe the environment and adapt its path.

2. Selected Technique:

Online Search Agent

This is more suitable than ordinary offline informed search because the robot does not know the complete environment in advance.

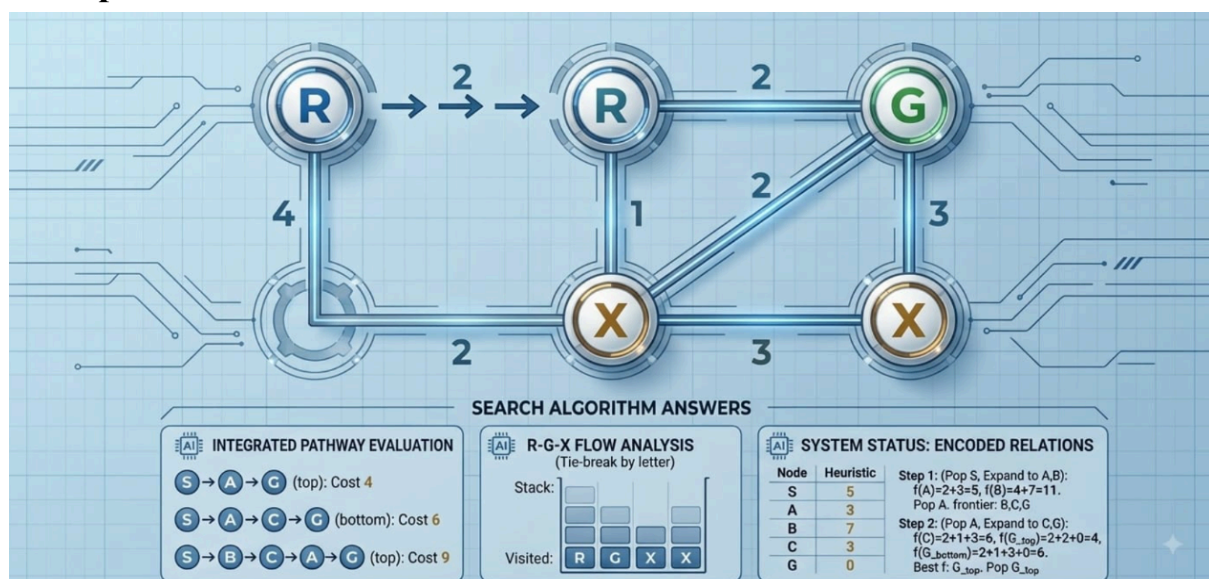
3. Initial State:

Robot → R

Goal → G

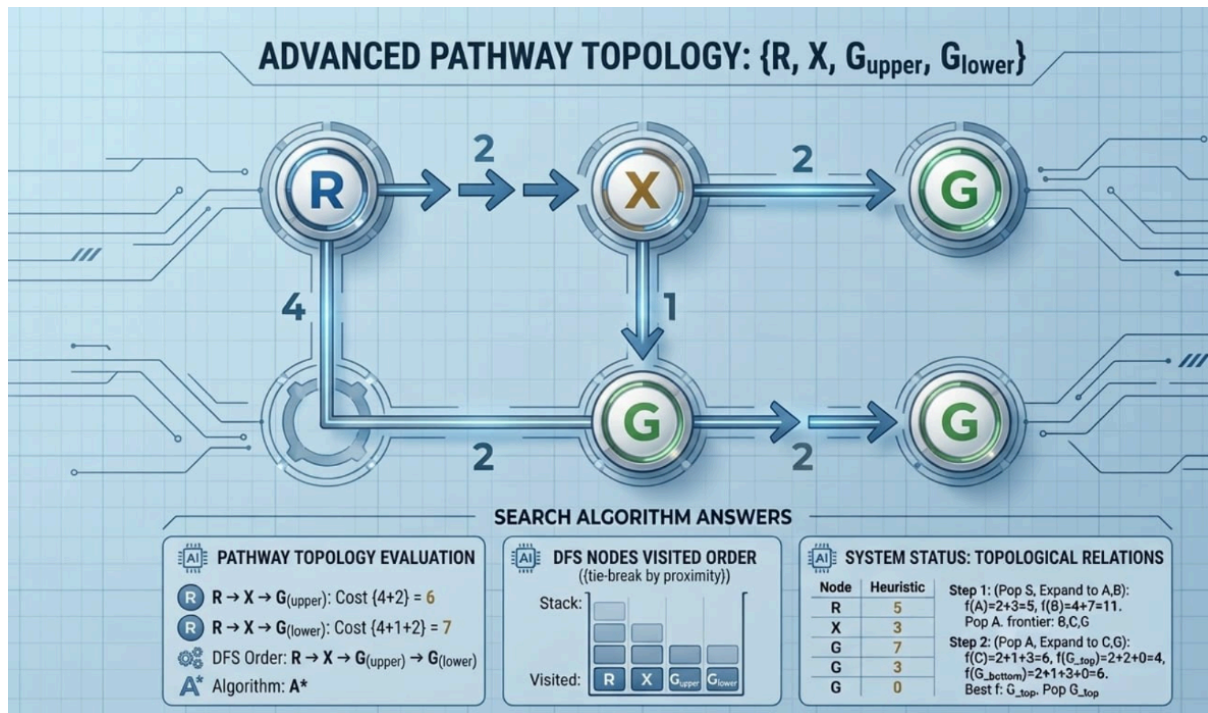
Environment → partially known

Example:



X represents an obstacle.

If a new obstacle appears:



The robot detects it and changes its route.

4. Online Search Strategy:

1. Observe current environment.
2. Identify available neighbouring states.
3. Select the most promising action.
4. Move to the selected state.
5. Observe the new environment.
6. Update the map.
7. If an obstacle blocks the current path:
Recalculate an alternative path.
8. Repeat until the goal is reached.

5. Pseudocode:

```
ONLINE-SEARCH(start, goal)

1. current ← start
2. learned_map ← empty

3. While current ≠ goal:
    | Observe environment
    | Update learned_map
    | If planned path is blocked:
    |   | Recalculate path
    |
    | Select best available action
    | Execute action
    |
    | current ← new position

4. Return SUCCESS
```

6. Adaptation to Environmental Changes:

Suppose the original route is:

R → A → B → C → G

An obstacle appears at B.

The robot detects:

B = blocked

It updates its knowledge and searches for:

R → A → D → E → G

Thus, the robot does not need to restart from the beginning.

7. Comparison with Informed Search:

Feature	Online Search	A*/Informed Search
Environment	Partially unknown	Usually known
Planning	During execution	Before execution
Handles new obstacles	Very well	Requires replanning
Adaptability	High	Moderate
Initial knowledge	Limited	More complete
Suitable for dynamic warehouse	Yes	Less suitable alone

8. Complexity:

The exact complexity depends on the search algorithm used for replanning.

For repeated graph search, worst-case time can approach:

$$O(bd)$$

and memory can be:

$$O(bd)$$

where b is branching factor and d is search depth.

10. Advantages:

- Adapts to unexpected obstacles.
- Learns about the environment while moving.
- Suitable for dynamic warehouse environments.
- Can continuously update the route.

10. Limitations:

- May take longer because decisions are made during execution.
- Replanning can increase computation.
- The robot may initially have incomplete information.
- Performance depends on how quickly environmental changes are detected.

11. Conclusion:

An Online Search Agent is appropriate for a warehouse robot because the environment is dynamic and unpredictable. The robot observes changes, updates

its knowledge, and replans its route whenever necessary. This provides greater adaptability than a fixed offline search strategy.